

Legacy Raspberry Pi Infrastructure Stabilization & Platform Migration

Redacted Production Case Study

Josh Heath

Linux Systems Administration • Edge Infrastructure • Manufacturing IT

Case-study purpose

To document the recovery, modernization, and ongoing administration of a long-lived Linux edge platform while preserving continuous manufacturing operations. The emphasis is on technical judgment, staged change, knowledge recovery, and maintainability rather than greenfield redesign.

Executive Summary

This case study describes primary operational ownership of a distributed fleet of 156 Raspberry Pi devices supporting MES access, label printing, production displays, operational dashboards, and warehouse terminals. The systems operate across multiple manufacturing facilities and serve approximately 200 production users in a 24-hour environment. The inherited platform had evolved through several hardware and operating-system generations, relied heavily on historical images and manual practices, and contained fragmented operational knowledge.

The work focused on restoring a supportable baseline without a full rewrite. Major efforts included rebuild-based Raspberry Pi OS migrations, removal of legacy NOOBS layouts, printer-platform integration, staged deployment and rollback practices, physical reliability improvements, documentation recovery, and the introduction of an Ansible environment for routine maintenance and monitoring.

150+

active edge devices

100+

Ansible-managed devices

~200

production users

24 hr

operational environment

Environment & Scope

Production functions supported

- MES access and operator workflows
- Raw IPL label printing
- Operational dashboards and displays
- Warehouse and production terminals

Platform characteristics

- Raspberry Pi 3B, 3B+, and 4B hardware
- Mixed Raspberry Pi OS releases
- Static network assignments
- USB-connected industrial label printers

Deployment model: Common master images, manually flashed storage, tracked static addresses, office validation, controlled production pilots, and batch rollout. Basic recurring maintenance and monitoring are now handled through Ansible.

Ownership: Primary administrator and maintainer of the fleet and sole administrator of the Ansible environment.

Inherited State

The fleet originated as a programmer-created production solution and remained in use after the original developer left the organization. Operational support then depended on older images, shell and Python scripts, manual post-deployment configuration, and knowledge distributed across incomplete notes and individual experience.

- Mixed operating-system releases and hardware generations with different 32-bit and 64-bit requirements
- NOOBS-based storage layouts that left approximately half of installed media unavailable to the active system on affected devices
- Unused or partly broken script paths that consumed resources or complicated diagnostics
- No original centralized configuration-management mechanism
- Printer replacement assumptions invalidated by a newer USB printer platform
- Recurring equipment damage caused by mobile use, impact, cable strain, heat, and inconsistent hardware handling
- Fragmented rebuild, maintenance, and diagnostic knowledge

Constraints & Engineering Principles

The environment could not be treated as a laboratory. Devices were in daily production use, deployment demand varied, and changes had to coexist with mixed platform states. A full outage was neither planned nor acceptable.

- Preserve operational continuity while changing platform foundations incrementally.
- Prefer rebuilds when in-place upgrades would retain unsupported layouts, permissions, or hidden assumptions.
- Validate behavior on a bench device, then on one live production device, before fleet rollout.
- Retain prior system images as named rollback baselines rather than deleting known-working states.
- Accept and document low-risk technical debt when correcting it would create disproportionate production risk.
- Treat physical design, operator workflow, and documentation as parts of infrastructure reliability.

Lifecycle-Driven OS & Hardware Migration

The documented platform history begins with Raspbian Jessie and progresses through Stretch, Buster, Bullseye, Bookworm, and Trixie. The fleet remains mixed because upgrades are introduced according to production demand, hardware compatibility, and validated application behavior rather than through a single disruptive cutover. Raspberry Pi 4 deployments began with Buster, and current Pi 4 builds use a 64-bit baseline.

Why rebuild instead of upgrade in place?

A direct upgrade path was not consistently safe across operating-system releases, hardware generations, filesystem layouts, and accumulated local modifications. Rebuilding from a controlled image allowed dependencies, configuration files, permissions, network details, branding, and required applications to be recreated deliberately rather than carrying forward unknown state.

- Created and maintained golden master images using Linux dd and Raspberry Pi Imager.

- Removed unused applications, libraries, language packs, wallpapers, and other nonessential components.
- Installed required applications and dependencies, recreated configuration files, and applied system identification and branding.
- Manually assigned static network settings and associated ERP print outqueues before release to production.
- Maintained a small stock of prebuilt devices and an updated pool of available network assignments for rapid replacement.

Validation, Pilot & Rollout Process

	Stage	Control
1	Bench build	Create or modify the image and validate configuration in the office.
2	Overnight test	Observe uptime, latency, paging accuracy, touch behavior, printing, and network stability.
3	Production pilot	Convert one device and monitor it in live use for one week.
4	Batch rollout	Deploy approved changes in groups of 10 devices.
5	Rollback readiness	Retain and clearly name the previous image as the pre-change baseline.

Operational result

Normal device swaps, including touch, printing, and network validation, can typically be completed in less than 10 minutes. Platform changes were introduced without a full-fleet outage.

Storage & NOOBS Remediation

Affected NOOBS-based layouts left approximately 50% of installed 16 GB, 32 GB, or 64 GB media unavailable to the active operating-system partition. The resulting space constraints contributed to failed updates, print-queue accumulation, and filesystem corruption. Attempts to remove NOOBS manually were not consistently reliable, so affected devices were rebuilt onto conventional images.

- Recovered approximately half of installed storage on affected devices.
- Removed a structural barrier to package maintenance and future migrations.
- Protected the boot partition from routine writes and restricted operator access to the desktop environment.
- Reduced SD-card writes by placing selected logs on volatile storage where appropriate for the appliance use case.
- Monitored disk capacity through message-of-the-day output and Ansible.

Printer Platform Migration

The original platform used Intermec/Honeywell PM43 and PX4i printers with raw IPL output. The established replacement model treated printers as interchangeable USB devices. When approximately 150 devices were migrated to Honeywell PM45 printers, that assumption no longer held because replacement units exposed serial-specific USB device URIs that had to match the CUPS queue configuration.

Implemented response

- Evaluated multiple printing approaches and retained a raw CUPS queue to preserve the existing IPL data path.
- Developed a shell utility that detected Honeywell/Intermec printer hardware URIs and updated the CUPS queue to reflect the connected printer serial number.
- Documented printer registration and replacement through ServiceDesk guidance with photographs.
- Kept upstream ERP print-outqueue administration separate from the Raspberry Pi transport layer; the edge device passes print data and does not control the ERP queue.
- Completed normal replacement configuration and validation in less than 10 minutes when IT intervention was required.

Known limitation

The shell utility is now treated as a legacy workaround. It reduced replacement effort, but it was never integrated into the operator interface. The case study retains it as evidence of practical adaptation, not as an ideal long-term design.

Ansible Introduction

The inherited environment initially had no centralized configuration-management mechanism. A recent improvement was the design and creation of an Ansible server for recurring maintenance and monitoring. I remain its sole administrator and maintain the server, access, inventory, playbooks, and variables.

- More than 100 production Raspberry Pi devices currently receive basic maintenance through Ansible.
- Disk-space visibility supplements local message-of-the-day checks.
- Major image creation, static addressing, hardware validation, and platform migrations remain intentionally controlled through the established provisioning process.
- The hybrid model reduces repetitive administration without pretending that the mixed legacy fleet is fully orchestrated.

Physical Reliability & Cost-Conscious Improvements

Recurring failures were traced to physical and environmental causes rather than software alone. Mobile production equipment exposed monitors, cables, printers, Raspberry Pi devices, and SD cards to impact, movement, heat, and incorrect hardware swaps.

- Installed fixed 45-degree mounting assemblies at approximately 150 production stations.
- Secured printers against rotation and mechanical strain.
- Labeled Raspberry Pi devices by network identity to improve identification and reduce incorrect swaps.
- Reduced frequent monitor falls and associated damage to uncommon incidents.

- Improved replacement consistency and troubleshooting speed without claiming unsupported dollar savings or uptime percentages.

Documentation & Knowledge Recovery

Approximately 10 documents were created or consolidated across Word, spreadsheets, and knowledge-base articles. The work covered deterministic rebuilds, maintenance, diagnostics, printer replacement, device identification, and operator-facing procedures.

- Converted fragmented and implicit system knowledge into repeatable rebuild and support guidance.
- Used simplified language, photographs, diagrams, arrows, and color coding for production audiences.
- Asked operators, team leaders, and third-party support personnel to review instructions before final release.
- Applied prior production-floor and team-leadership experience to make the documentation match actual workflows.
- Enabled support staff to perform selected diagnostic and replacement procedures without direct assistance.

Current Support Model

Automated or centrally supported

- Routine Ansible maintenance for 100+ devices
- Disk-space monitoring
- Common image baselines
- Archived rollback images and configuration copies

Manually controlled

- SD-card imaging and final configuration
- Static IP assignment and ERP queue association
- Bench and production validation
- Hardware and printer replacement

Results

- Kept the 156-device platform in continuous production while introducing OS, hardware, storage, printer, and management changes.
- Established a supportable migration path across Raspberry Pi OS releases from Jessie through Trixie and Raspberry Pi 3B, 3B+, and 4B hardware.
- Replaced unreliable NOOBS-based layouts and recovered approximately half of installed storage on affected devices.
- Integrated approximately 150 PM45 printers into the existing raw IPL workflow through serial-aware CUPS configuration.
- Introduced staged testing, one-device pilots, 10-device rollout batches, rapid replacement stock, and named rollback baselines.
- Built an Ansible environment for routine maintenance and monitoring across more than 100 devices.
- Reduced recurring physical equipment damage through fixed mounting, printer restraint, and device labeling.
- Reconstructed operational knowledge so the platform can be rebuilt, diagnosed, and supported more consistently.

Lessons Learned

Legacy systems degrade quietly: Unsupported images, small storage failures, obsolete peripherals, and undocumented scripts may remain usable until several assumptions fail at once.

Rebuilds can be safer than upgrades: A clean, controlled image may reduce risk when an in-place upgrade would retain unknown permissions, layouts, dependencies, and historical errors.

Peripheral changes can alter architecture: A printer replacement changed USB identity and queue behavior, invalidating an established drop-in replacement model.

Physical design is infrastructure work: Mounting, cable strain, mobility, heat, and labeling directly affect availability and support cost.

Automation should fit the environment: Ansible adds value for recurring maintenance, while controlled imaging and staged rollout remain appropriate for major changes in a mixed fleet.

Documentation must be tested by its audience: Instructions become more useful when operators and support staff validate them against actual production workflows.

What This Case Study Demonstrates

Ownership

Primary stewardship of an inherited production platform

Judgment

Controlled change under operational constraints

Breadth

Linux, Ansible, networking, printing, hardware, and documentation

- Linux systems administration across mixed operating-system and hardware generations
- Configuration management and monitoring through a self-built Ansible environment
- Golden-image creation, staged deployment, production pilots, rollback planning, and rapid replacement practices
- Root-cause analysis spanning software, storage, networking, peripherals, and physical equipment
- Pragmatic modernization without a disruptive rewrite
- Knowledge preservation and audience-reviewed technical documentation

Scope, Confidentiality & Accuracy

This public case study is independently written and redacted. It does not include employer source code, system images, credentials, hostnames, IP addresses, internal screenshots, production data, Ansible inventories, ERP queue names, or proprietary procedures. Counts describe operational scale and may be rounded in public portfolio versions. Results are stated using observable outcomes and process evidence rather than fabricated KPI, uptime, or savings figures.

Relationship to Other Portfolio Work

Lessons from this fleet informed the Linux Thin-Client Architecture and PySide6 UI Prototype. That independent exercise explores how a future edge terminal might reduce dependencies, separate operator and administrative functions, and make deployment assumptions explicit. It is presented as a prototype and architectural study, not as a completed replacement for this production system.

Closing Statement

The outcome was not a perfect or fully automated platform. It was a substantially more understandable, supportable, and resilient production system, improved through deliberate migration, operational testing, practical automation, physical reliability work, and documentation that can outlast individual ownership.

Josh Heath • Linux Systems Administrator | Edge Infrastructure | Manufacturing IT

Josh Heath • Redacted Raspberry Pi Infrastructure Case Study